

СУ “Св. Климент Охридски”, ФМИ
курс - Системи за паралелна обработка

Hello-WaTor

Многонишкова симулация на хищници и плячка в
торовиден свят

Цветомир Стайков
1MI0800469
Компютърни науки
3 курс 2 група

Май 2026

Съдържание

Въведение.....	2
Увод.....	2
Реализация.....	2
Технологии.....	2
Машини.....	2
Симулация.....	3
Поле.....	3
Структура на проекта.....	4
Изпълнение.....	5
Агенти в симулацията.....	6
Риба.....	6
Акула.....	6
Алгоритъм.....	6
Сектори.....	7
Синхронизация.....	9
Случайности.....	9
Параметри.....	9
Визуализация.....	10
Сравнение с модели.....	10
Други проучвания и проекти.....	11
Резултати и анализ.....	12
Таблица.....	13
Графики.....	14
Заклучение.....	15
Многонишковы програми.....	15
Синхронизация.....	15
Източници и подобни проучвания.....	16
Статии.....	16
Подобни проучвания.....	16
Допълнителна информация и материали.....	16

Въведение

Увод

Този документ представя реализация на симулация на хищници и плячка по модела на А. К. Dewdney, представена в статията от 1984 "Computer Recreations: Sharks and fish wage an ecological war on the toroidal planet Wa-Tor" [1]. Ние ще наричаме плячката - риби, а хищниците - акули. Разработен е проект, симулиращ правилата на симулацията. Програмата е реализирана в нишков модел. Обработват се в стил *SPMD (Single program, multiple data)*. Представени са данните от изпълнение. Фокус ще се отдели върху сравнение на различен брой нишки, върху един и същ размер поле, с цел да се покаже паралелната работа на една система и ефективността на различен брой нишки.

Реализация

Технологии

Кодът на симулацията е написан на **C++20** с компилатор **g++ 15.2.1**. Допълнителни библиотеки са **SFML** за визуализация и **STL** за стандартни структури от данни. Графиките са създадени от **Python 3.14** скриптове с **Matplotlib**.

Машини

Кода е писан и тестван на лаптоп Thinkpad L13 Yoga с Linux 7.0.9 Fedora 43 64-bit, но финалните резултати са на машина със следните спецификации:

Architecture:	x86_64
CPU op-mode(s):	32-bit, 64-bit
Byte Order:	Little Endian
CPU(s):	32
On-line CPU(s) list:	0-31
Thread(s) per core:	2
Core(s) per socket:	8
Socket(s):	2
NUMA node(s):	2
Vendor ID:	GenuineIntel
CPU family:	6
Model:	45

Model name:	Intel(R) Xeon(R) CPU E5-2660 0 @ 2.20GHz
Stepping:	7
CPU MHz:	3000.000
CPU max MHz:	3000.0000
CPU min MHz:	1200.0000
BogoMIPS:	4389.50
Virtualization:	VT-x
L1d cache:	32K
L1i cache:	32K
L2 cache:	256K
L3 cache:	20480K
NUMA node0 CPU(s):	0-7,16-23
NUMA node1 CPU(s):	8-15,24-31

Симулация

Симулираният терен представлява двуизмерно поле от клетки. Всяка клетка може да съдържа риба (*зелен цвят*), акула (*червен цвят*) или празно място (*черен цвят*). Времето се измерва в единицата хронони (*chronons*), като често ще ги наричаме стъпки (*един хронон = една стъпка*).

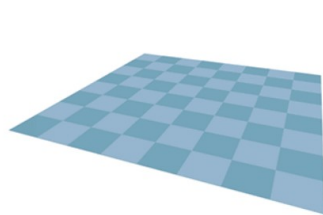
Рибите и акулите, наричани още агенти, спазват основни правила за придвижване. Освен тях са дефинирани и допълнителни правила, които могат да се променят чрез параметри.

Поле

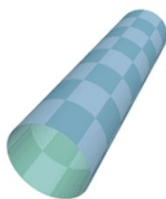
За една стъпка може да се премести агент от една клетка в съседна до нея, като се движи хоризонтално или вертикално със само една стъпка. При достигане на единия край на полето, се преминава до другия му край. Тоест ако се придвижим нагоре и сме на горния ръб, ще отидем при долния ръб и обратното. Аналогично за ляво и дясно.

По-формален модел: Ако имаме редове от 1 до R и колони от 1 до C, където R и C са естествени числа. Ако се придвижим от ред 1 нагоре, ще се преместим на ред R. Ако се придвижим от ред R надолу, ще се преместим на ред 1. Ако се придвижим от колона 1 наляво, ще се преместим на колона C. Ако се придвижим от колона C надясно, ще се преместим на колона 1. За всеки друг ред или колона просто ще добавим плюс или минус 1 към преместването.

Това правило кара света да описва торус (или поничка), като в оригиналната статия го представят по следния начин - „Планетата Wa-Tor... е под формата на торус, поничка, покрита изцяло с вода."



Фигура 1: Просто поле



Фигура 2: Цилиндър



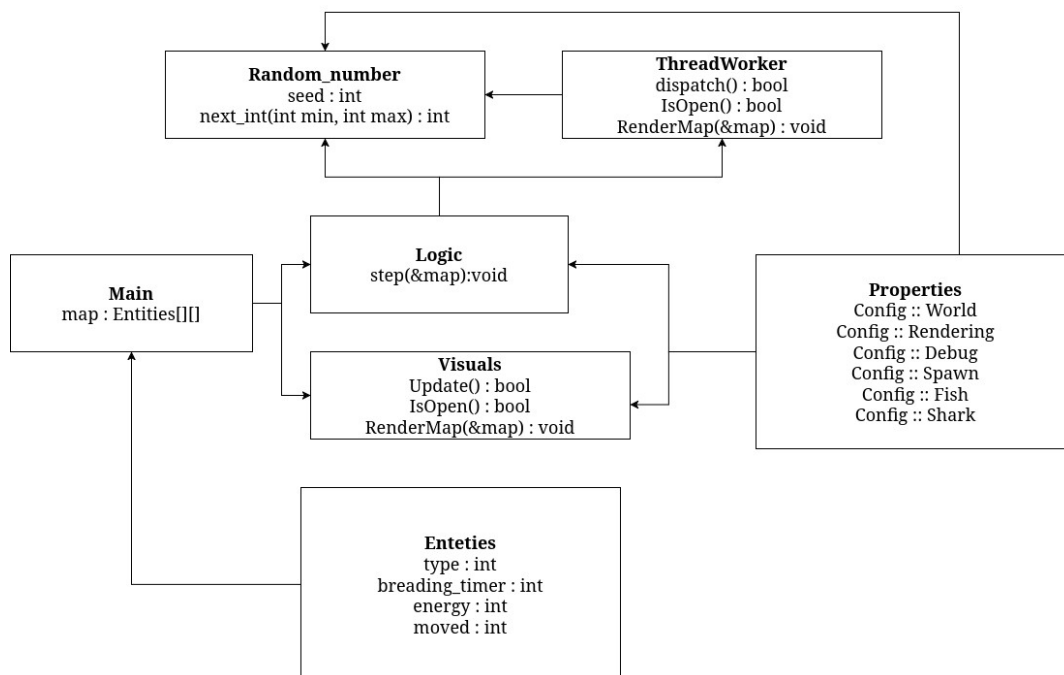
Фигура 3: Торус

Може да си представим полето (фигура 1) като лист хартия и да го огънем в цилиндър (фигура 2). Ще имаме два отворени края и ако ги съединим, ще получим торус (фигура 3). По този начин свързваме ръбовете на полето, спазваме описаните правила за движение и образуваме фигурата. [9]

Структура на проекта

Проектът е съставен от 6 различни обекта.

- Visuals – визуализация на полето
- Entities – съхраняване на информация за рибите и акулите
- Random Numbers – генератор на случайни положителни цели числа
- Thread Worker – система за нишки и обработване на задачи
- Logic – основна логика на симулацията
- Properties – параметри за симулацията, визуализацията и изходните данни



Фигура 4: Класове и обекти в симулацията

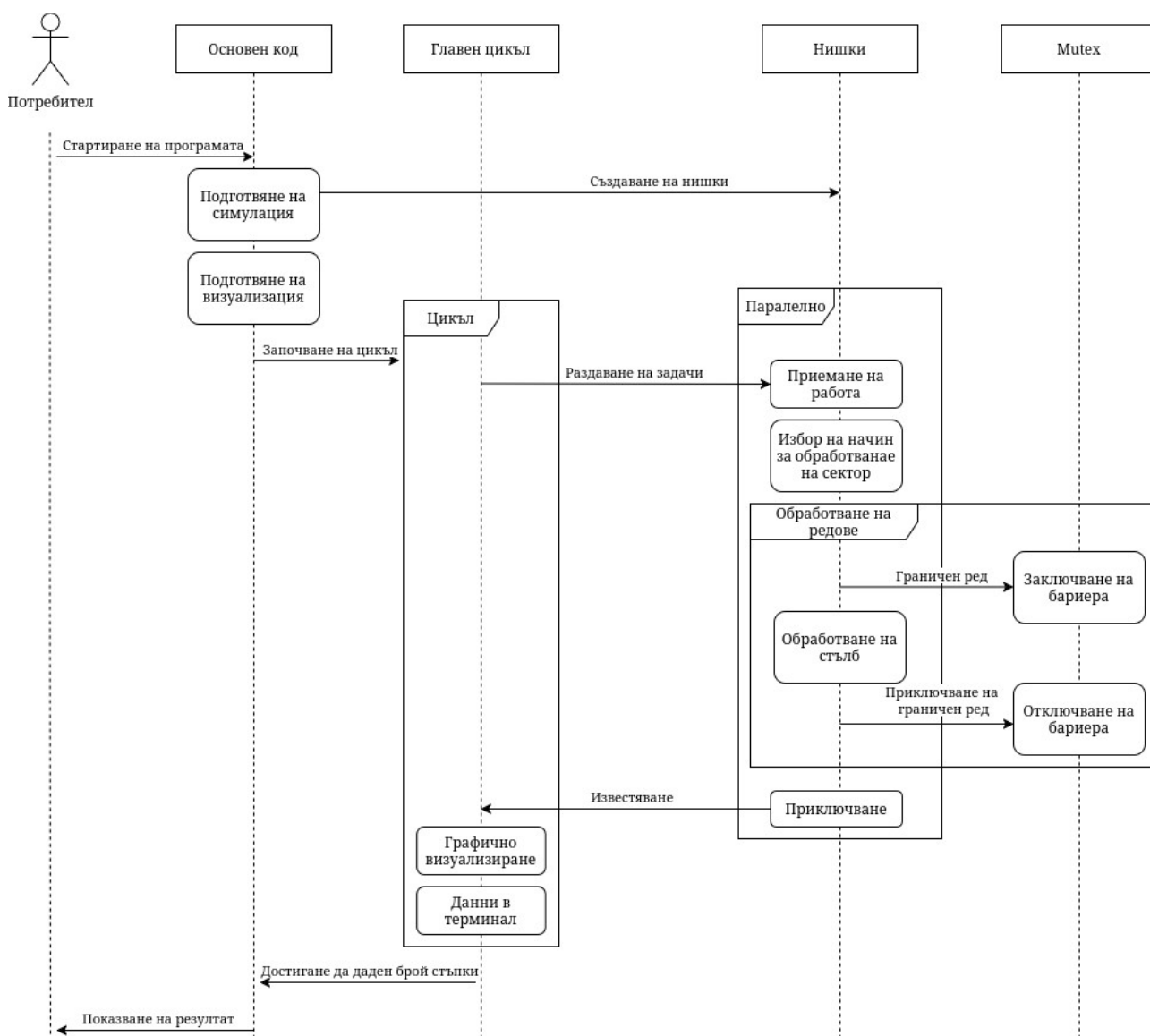
Изпълнение

Програмата следва прост път на изпълнение. Първоначално се подготвят нужните ресурси – масив за полето, нишките и визуализацията. След това програмата влиза в цикъл, където се задава работа на всяка нишка и след това се изчакват да приключат.

След приключването на всички нишки, процесът продължава в цикъл, докато желаният брой стъпки не е достигнат или докато потребителят не спре симулацията. Този основен цикъл наричаме **оркестратор**, а другите наричаме **работещи нишки** (*Thread workers*).

Всяка нишка чака, докато не получи работа. След това започва да обработва обозначения за нея сектор. При приключване на работата си, тя информира оркестратора и чака за нова задача.

Цялата синхронизация се случва чрез *mutex* и *condition variables*.



Фигура 5: Изпълнение на симулацията

Агенти в симулацията

За всяко правило, до което пише „(конфигурируемо)“, има параметър в *parameters.hpp*

Риба

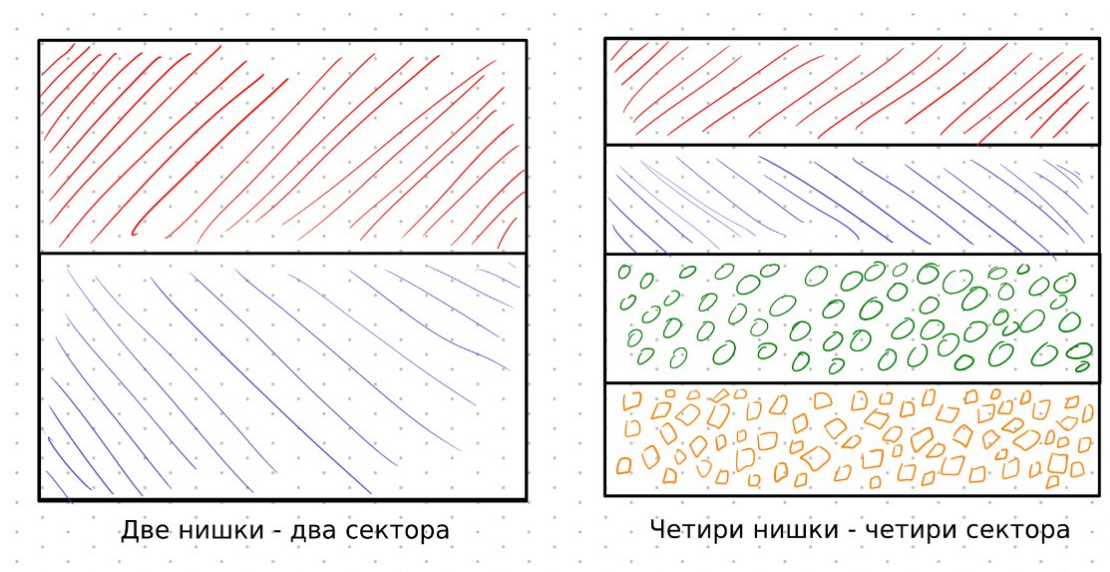
- За една стъпка, рибата се мести случайно в съседна празна клетка. Ако няма такава, си остава на същата клетка.
- Ако рибата оцелее даден брой стъпки, тя може да се размножи. Това става като се премести в съседна клетка, а нова риба се появява на старото ѝ място. Започват да броят стъпките за размножаване от 0 и за двете риби. Ако рибата не може да се премести, не се размножава. Стъпките се броят като таймер. (конфигурируемо)
- Рибите не умират, освен ако не са изядени от акула.

Акула

- За една стъпка, акулата се премества до съседна клетка с риба, при което я изяжда. Ако няма риби наоколо, се мести до празна клетка. Ако няма празна клетка, остава на същото място.
- При старт се започва с определен брой енергия. (конфигурируемо)
- На всяка стъпка акулата губи енергия. (конфигурируемо)
- При достигане на нулева енергия, акулата умира.
- Ако акулата изяде риба, получава енергия. (конфигурируемо)
- Ако акулата достигне даден брой стъпки оцеляване, тя може да се размножи. Новата и старата акула започват с първоначалната енергия от старта на симулацията. (конфигурируемо)

Алгоритъм

Симулацията изпълнява **SPMD (Single program, multiple data)** като се възползва от нишки. *Single Program*, защото един тип алгоритъм и програма се изпълнява. *Multiple Data*, защото разделяме полето на няколко части от един и същ тип. Тези части накрая се обединяват в една голяма матрица. Използва се само един буфер, а агентите отбелязват четност, за да не се преместват два пъти по време на изпълнение. Броят на нишките се избира от глобален файл с параметри *properties.hpp*. Полето се разделя на равни сектори спрямо броя на нишките. Ако има остатък при делението, програмата ще спре. Така всяка нишка има еднакъв брой редове и ще имат приблизително еднакво време на приключване.



Фигура 6: Разпределение на полета спрямо нишки

Всеки сектор се обработва от една нишка. Нишките не си сменят секторите. Това се класифицира като **статично балансиране**.

Основната логика се намира в *1b-logic.hpp*. Оркестрирането и основният цикъл са в *main.cpp*. Процесът е следният:

Извикване на нишките

Изчакване на нишките да приключат

Показване на време за изпълнение

Всяка нишка изпълнява следния код:

Избиране на начин на обхождане спрямо четност на времето

Обхождане на редове:

Заклучване на мютекс ако обхождаме ред около ръба на сектора

Обхождане на всяка колона от реда

Ако е риба

местим риба

Ако е акула

местим акала

Алгоритъмът има **статична декомпозиция**. Всеки сектор е раздаден на дадена нишка. Това позволява детерминираност, когато една нишка обработва само един сектор.

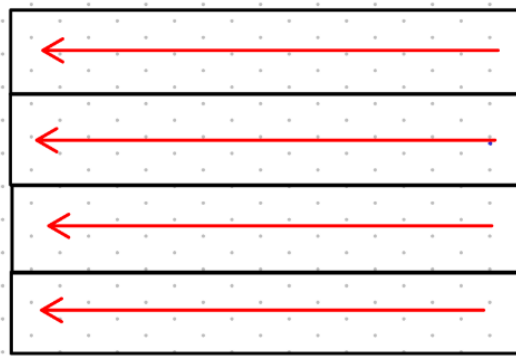
Сектори

Секторите представляват определен брой редове, без прекъсване между редовете или в редовете. Това дава възможност за по-проста логика, ясно разделение между нишките и по-малко мютекси спрямо други реализации (например шахматна дъска).

За избягване на специфични повторения и за по-органично изглеждащо движение на агентите, е имплементирано променяне на реда на обработка на клетките. Това се определя спрямо стъпката и нейната четност при $\text{mod } 4$.

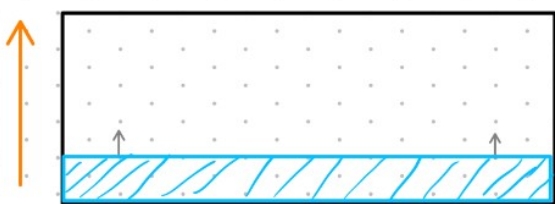
Даден сектор се обработва по различни начини. При четна стъпка се обработва отгоре надолу, а при нечетна — отдолу нагоре. Редуват се обработване отляво надясно и отдясно наляво. Цикълът изглежда по следния начин:

1. Отгоре-надолу, отляво-надясно
2. Отдолу-нагоре, отляво-надясно
3. Отгоре-надолу, отдясно-наляво
4. Отдолу-нагоре, отдясно-наляво

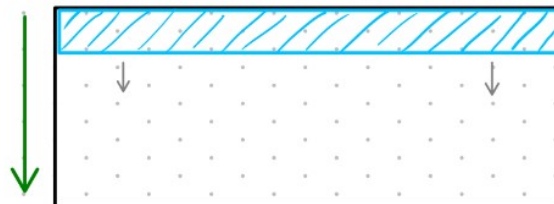


Алтерниране на обработка по редове

Фигура 7: Алтерниране по редове спрямо четността



четна стъпка - отдолу-нагоре



нечетна стъпка - отгоре-надолу

Фигура 8: Алтерниране на редове спрямо четността

Благодарение на това, нашата система изглежда много по-случайна. Избягват се повторения и образуване на специфични движения. Същата симулация със стандартно движение отгоре надолу, отляво надясно създава ефект на изплуващи балончета, което не изглежда като случайно движение.

Синхронизация

Основен проблем при работата с нишки е синхронизацията. В този проект нишките комуникират със своите съседи (*нагоре и надолу*). За избягване на състезателен достъп (*race condition*) се използват mutex системи за заключване на границата между две нишки за кратко време. Това осигурява възможност на една нишка да работи, без да променя данните на друга случайно.

Тези бариери покриват общо 4 реда, 2 от едната нишка и 2 от другата. При работа на ръба на даден сектор, една нишка може да се наложи да прехвърли една клетка в друг сектор. При това съседната нишка не трябва да модифицира реда, на който нашата нишка работи. Разбира се, това не е перфектно решение, защото заключваме цял ред на друга нишка, дори ако не прехвърляме клетки или прехвърляме много малко, но така кодът е доста по-прост, има по-малко mutex заключвания и знаем, че ако една нишка приключи и освободи реда, другата може да работи на него, без да се налага първата да се върне или да чака.

Заклучваме 2 реда, защото предпоследната редица може да прехвърли клетка към последната редица, а съседната нишка може да прехвърли към нашата нишка един от своите агенти, при което ще имаме колизия.

За да се избегне прехвърляне и пренаписване на клетки една върху друга, всяка клетка има брояч, който следи четността на стъпката. Така ако една клетка има същата четност като текущото време, тогава тя вече е била променена и е забранено да се променя отново.

Случайности

При избиране на случайности се използва отделен клас и проста математика. Прилага се код за случайност на симулацията (*seed*), който се намира в *properties.hpp*. Спрямо този код се създават случайни числа. Алгоритъмът е базиран на **xorshift** [3]. За целите на този проект е имплементирано само **int next_int(int min, int max)** и се използва само за положителни числа. Всяка нишка получава собствен клас за случайни числа със специфичен код за случайност, съставен от номера на нишката и оригиналния код за случайност.

Алгоритъмът осигурява, че всяко пускане на симулацията без промяна на параметрите ще върне една и съща симулация. Разбира се, времето на изпълнение може да е различно в зависимост от компютъра, ресурсите и много други фактори.

Параметри

Програмата предоставя различни параметри в няколко категории:

- **World** – параметри за размер на прозореца, резолюция на визуализация, изчакване между всяка стъпка, брой стъпки и брой нишки. Ако броят стъпки е 0, симулацията няма да спре и ще работи, докато не бъде прекъсната от потребителя или друг процес.
- **Rendering** – параметри за визуализацията, цвят на агентите и кадри в секунда.
- **Debug** – дали да записва във файл, дали да брое животните в конзолата, дали да направи няколко симулации с изходни параметри, дали да визуализира.

- Spawn – колко риби и акули да има в симулацията при стартиране.
- Fish – правила за рибите: таймер за размножаване.
- Sharks – правила за акулите: започваща енергия, колко енергия да получи при хранене, губене на енергия при стъпка, нужна енергия за размножаване.

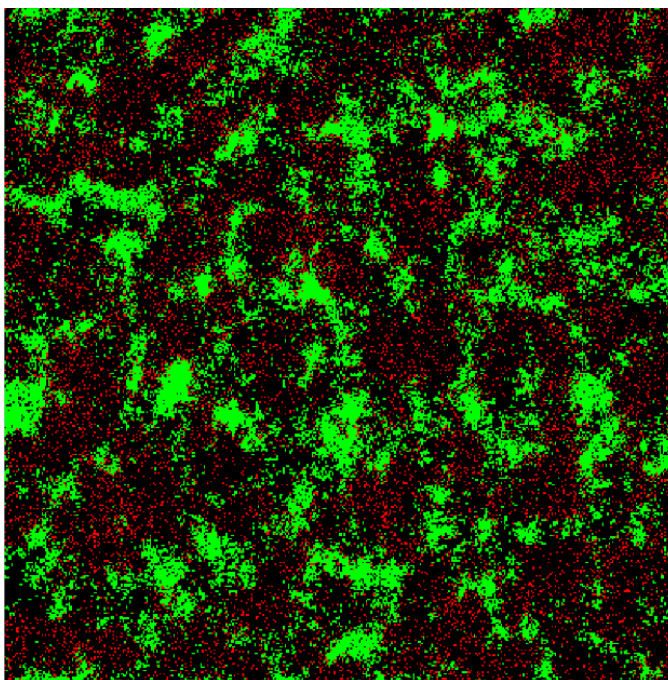
Визуализация

За визуализацията се използва библиотеката SFML. При отворен прозорец може да се паузира симулацията чрез бутона **Space**, след което да се използва **стрелката надясно** за продължение с една стъпка. Чрез бутона **Escape** може да приключим симулацията и да изведем резултат в терминала.

Визуализацията не зависи от размера на полето. Тя се адаптира спрямо зададената резолюция на екрана. При поле 100x100 и резолюция 1000x1000, една клетка ще бъде 10x10 пиксела.

Резолюцията не се променя спрямо размера на прозореца, а спрямо параметрите в World. (вижте раздел [Параметри](#))

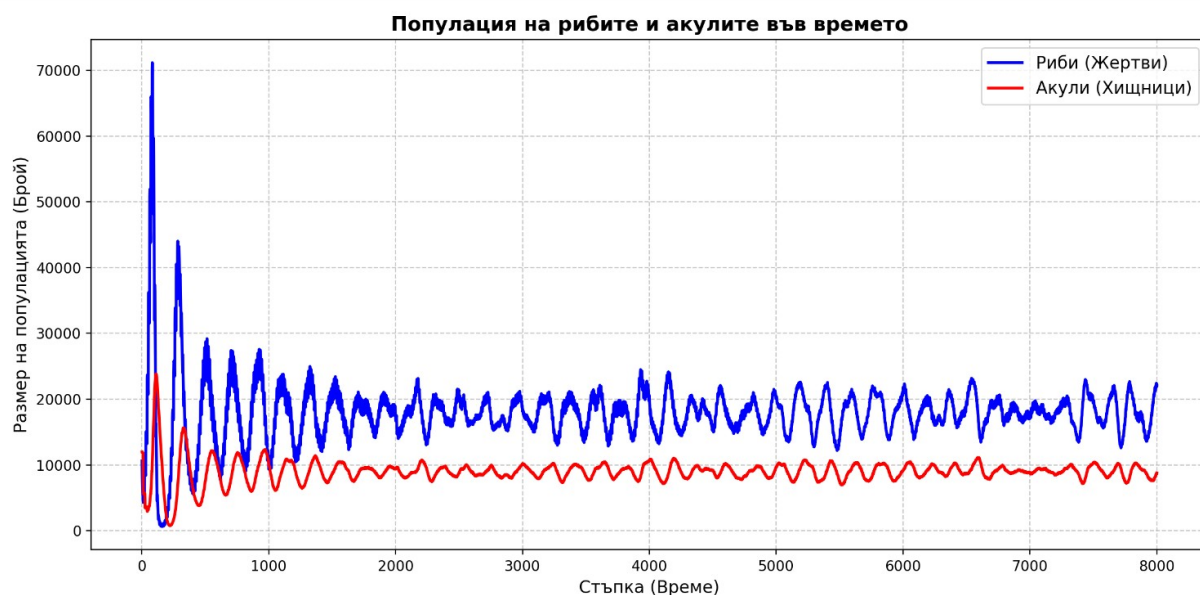
Цветовете на рибите, акулите и празните клетки могат да се променят в параметрите



Фигура 9: Визуализация 400x400 поле с резолюция 1000x1000

Сравнение с модели

След записване в файл на броя на рибите и акулите в даден момент, може да направим анализ. Виждаме осцилиране на популацията на рибите и акулите, което е очаквано спрямо модели като този на Лотка-Волтера за изчисление на плячка и хищници [2] [8]. В даден момент рибите стават повече, при което акулите имат повече храна и ги изяждат. При повече акули рибите намаляват и вече няма храна за акулите, след което те умират. След това рибите нямат опасност и могат да се размножат. Така цикълът продължава. В началото виждаме голям скок в броя на рибите и акулите. Този ефект е причинен от равномерното поставяне на рибите и акулите в началото на симулацията.



Фигура 10: Популация на риби и акули



Фигура 11: Брой риби спрямо акули

Разбира се, може да има случай, в който единият вид надделее над другия и рибите бъдат изядени, при което акулите ще умрат от липса на храна, или акулите ще умрат и рибите ще се размножават, докато не запълнят екрана.

Други проучвания и проекти

Wa-Tor е добър пример за система за паралелна обработка. Има много начини да се реализират алгоритмите и синхронизацията на нишките. Ще сравним няколко проекта на колеги, дадени като примери в курса по Системи за Паралелна Обработка. [4] [5] [6]

	Иванов [4]	Чакъров [5]	Караджова [6]	Hello-Wator
Модел	SPMD	Master-slave	MPMD	SPMD
Декомпозиция	Шахмат	По редове	По колони	По редове
Балансиране	Статично	Статично	Статично	Статично
Синхронизация	Четно-нечетно следене	Семафори	Мютекс	Мютекс
Технология	C#	Java 16	Golang	C++ 20
Ускорение*	2	3.8	3.1	3.8
Размер на поле	1920x1080	1000x2000	980x1920	1920x1920

*Ускорението е разликата между 1 и 4 нишки.

Можем да видим различни имплементации на симулацията. Разбира се, това не е точно сравнение, защото разглеждат различни размери на полетата, но са достатъчно близки. Въпреки това можем да видим алтернативни възможности за имплементация на проекта.

Използването на четно-нечетно синхронизация на редовете и подаването им на нишки, би било най-оптималното решение. Това ще позволи премахването на мютекси и би ускорило работата на програмата. В резултат бихме имали по-голяма производителност при по-висок набор от нишки. Изборът на шахматен дизайн за обработка е доста интересен, но бил донесъл проблем при четно нечетно следене. Заради ръбовете на бариерите, ако не е имплементирано следене на четността на стъпките при местенето на животните, бихме имали пренаписване на данни в една клетка. Това би се случило в ръба на всяко квадратче на шахматната дъска.

Използването на семафори би било ненужно. Те имат възможност да синхронизират много процес по по-сложен начин от този на мютекс, който има само 2 състояния. За този проект не би било достатъчно подходящо, но би било полезно при шахматната декомпозиция.

Тук можем да видим, че използването на мютекс е свело проект [6] и описания в този документ до почти еднакво време на изпълнение, въпреки различната технология и размер на полето. Декомпозицията по редове и стълбове не би трябвало да доведе до различно времева сложност, но ако масивите не са написани правилно може да имаме доста голяма честота на кеш пропускания, която да намали производителността.

Резултати и анализ

Направени са 5 теста за избран брой нишки – 1, 2, 3, 4, 5, 6, 8, 10, 12, 15, 16, 20, 24, 30, 32. По време на тестовете не се визуализира или записва нищо. Премахнати са изчакванията. Така ще получим максимални скорости.

Параметри на симулацията:

General	WAIT_SECONDS_BEFORE_NEXT_STEP	0
General	GRID_SIZE_WIDTH	1920
General	GRID_SIZE_HEIGHT	1920

General	SEED	1235
General	STEPS	2000
Spawn	INITIAL_FISH_DENSITY	0.05
Spawn	INITIAL_SHARK_DENSITY	0.05
Fish	STARTING_BREED_TIMER	10
Shark	STARTING_ENERGY	10
Shark	EATING_GAIN_ENERGY	15
Shark	STEP_LOSE_ENERGY	1
Shark	BREED_ENERGY_REQUIRED	90

Таблица

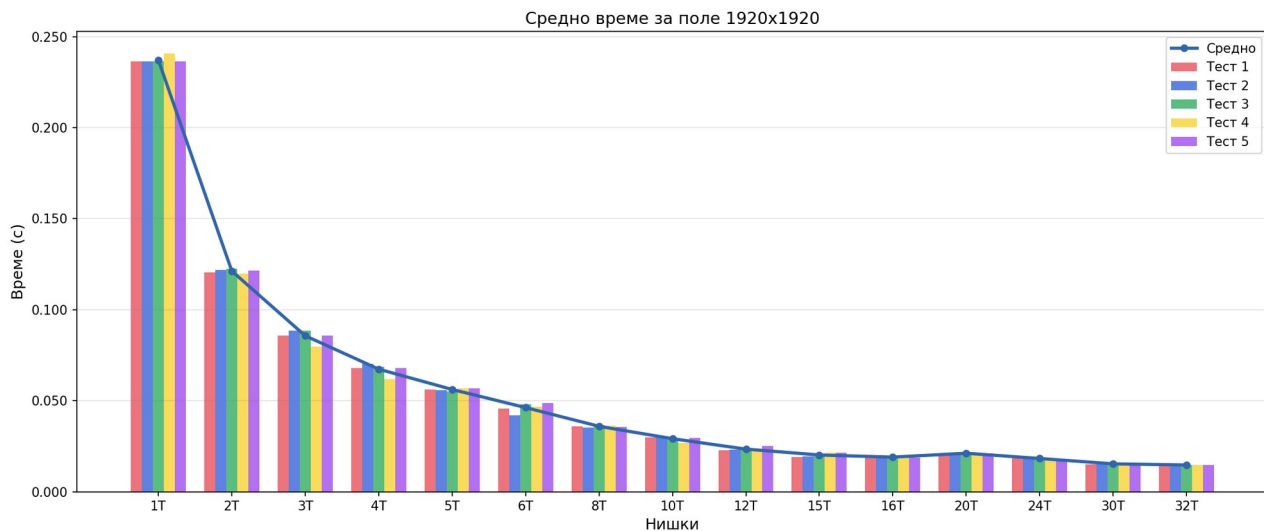
Следната таблица може да се чете по следния начин:

- # - номер на типа експеримент
- p – броят нишки
- g – гранулярност - броят редове за дадена нишка
- $T_p^{(i)}$ – средно време за изпълнение на една стъпка, за експеримент i . Общо има 5 експеримента пуснати за даден брой нишки p .
- T_p - най-бързо време от всички 5 експеримента.
- S_p - ускорение спрямо времето за 1 нишка.
- E_p - ефективност спрямо броя нишки

#	p	g	$T_p^{(1)}$	$T_p^{(2)}$	$T_p^{(3)}$	$T_p^{(4)}$	$T_p^{(5)}$	$T_p = \min(T_p^{(i)})$	$S_p = T_1/T_p$	$E_p = S_p/p$
1	1	1920	0.236302	0.23642	0.236292	0.240895	0.236313	0.236292	1	1
2	2	960	0.120417	0.121888	0.122671	0.119692	0.121527	0.119692	1.974167	0.9870835
3	3	640	0.085741	0.088551	0.088532	0.079787	0.085828	0.079787	2.9615351	0.9871784
4	4	480	0.068056	0.070375	0.06849	0.061734	0.068081	0.061734	3.8275829	0.9568957
5	5	384	0.056148	0.055747	0.055262	0.056813	0.056933	0.055262	4.2758496	0.8551699
6	6	320	0.045742	0.042009	0.047967	0.046853	0.048582	0.042009	5.6247947	0.9374658
7	8	240	0.035813	0.035345	0.036681	0.036209	0.035678	0.035345	6.685302	0.8356628
8	10	192	0.029815	0.030231	0.02932	0.026751	0.029606	0.026751	8.8330156	0.8833016
9	12	160	0.022838	0.02299	0.023128	0.022865	0.025236	0.022838	10.34644	0.8622033
10	15	128	0.019199	0.019487	0.019394	0.021416	0.021374	0.019199	12.307516	0.8205011
11	16	120	0.0193402	0.018707	0.019563	0.018827	0.018683	0.018683	12.647433	0.7904646
12	20	96	0.021132	0.021137	0.021104	0.02112	0.021173	0.021104	11.19655	0.5598275
13	24	80	0.018252	0.018253	0.018276	0.018345	0.018218	0.018218	12.970249	0.5404271
14	30	64	0.0150538	0.0154838	0.0154511	0.0154949	0.0150995	0.0150538	15.696502	0.5232167
15	32	60	0.014651	0.014553	0.014791	0.01458	0.014801	0.014553	16.236652	0.5073954

Таблица 1: Тестове и скорости между всяка нишка;
5 теста за всяка бройка нишки

Графики

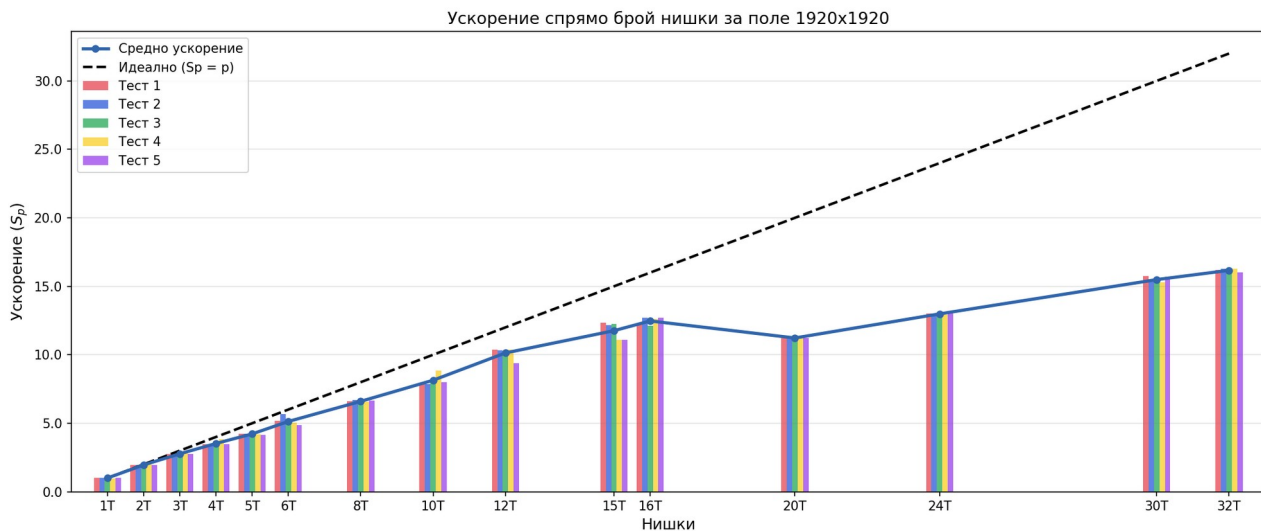


Фигура 12: Визуализация на таблицата – брой нишки, време на изпълнение;
5 теста за всяка нишка

На фигурата може лесно да се забележи, че добавянето на повече нишки ускорява симулацията и намалява времето между изпълнението на дадени стъпки. В началото времето се съкращава наполовина, но впоследствие това ускорение се забавя. В следващите диаграми може да видим закона на Амдал.



Фигура 13: Измерване на ефективността (E_p) за всеки тест



Фигура 14: Измерване на ускорението (S_p) за всеки тест

Разбира се, въпреки че времето се ускорява, ефективността не е винаги еднаква. Резултатите показват срещана тенденция при използване на нишки и това е намаляването на ефективността. До 15 нишки намираме над 80% ефективност. След това има спад, като накрая при 32 нишки намираме намаляване на ефективността до 50%. Забележимо е, че при 20 нишки имаме спад на ефективността спрямо околните тестове. Това може да се дължи на изчерпване на физически ядра, неоптимизации на кеша, споделяне на ресурси между двата процеса на системата и/или други подобни проблеми. При повтаряне на експеримента със същия брой нишки, резултатът отново остана същият, което премахва възможността машината да е била фокусирана върху друг процес, тъй като е споделена между няколко потребителя.

Заклучение

Многонишковите програми

Използването на няколко нишки може да ускори процеса на работа на дадена програма, но ефективността и ускорението няма винаги да са перфектни. В някои случаи използването на повече нишки може да доведе до по-лош резултат заради начина, по който данните се предават.

Въпреки това те са основна и важна част от съвременното програмиране и правилното им използване би ускорило една програма многократно, спрямо алгоритмите и синхронизацията.

Синхронизация

Споделянето на памет между нишки може да е сложен процес. За избягване на състезателен достъп се използват различни техники – мютекс, семафори, spinlock и други.

Разбира се, голяма гранулярност и неподходящо имплементиране може да доведат до повече проблеми, отколкото решават. Например ако имаме 1920 мютекса или дори 480 мютекса,

една нишка ще заключи две нишки за доста дълъг период от време, а отключването и заключването постоянно би изисквало повече ресурси, отколкото освобождава.

Този процес би бил по-лош от това да се използва само 1 нишка. Всяка нишка ще изчаква другата и накрая системата ще обработва данните както една нишка би ги обработила, но допълнително ще има операции за отключване и заключване на дадени сектори.

Източници и подобни проучвания

Статии

[1] Dewdney, Alexander Keewatin. Sharks and fish wage an ecological war on the toroidal planet Wa-Tor. Scientific American, 1984.

[2] Anisiu, Mira-Cristiana. LOTKA, VOLTERRA AND THEIR MODEL. DIDACTICA MATHEMATICA, Vol. 32, 2014, pp. 9–17

[3] Marsaglia, G. Xorshift RNGs. Journal of Statistical Software, vol. 8, no. 14, pp. 1–6, Jul. 2003.

Подобни проучвания

[4] Веселинов Чакъров, Иван-Асен. Изследване на скалируемостта на Wa-Tor симулацията при статично балансиране. 7 Юли 2021.

[5] Иванов, Александър Иванов. Паралелизирана симулация на популационна динамика „Wa-Tor“. Юли 2021.

[6] Караджова, Ива. Wa-Tor Проект за курса „Системи за паралелна обработка“. 7 юли 2021.

Допълнителна информация и материали

[7] Muñoz, Inigo. Wator — имплементация и въведение. (<https://beltoforion.de/en/wator/>)

[8] Wolfram Research. Калкулатор на Хищник-Плячка модела. (<https://demonstrations.wolfram.com/PredatorPreyModel/>)

[9] Wikipedia. Torus — Flat torus. (https://en.wikipedia.org/wiki/Torus#Flat_torus)